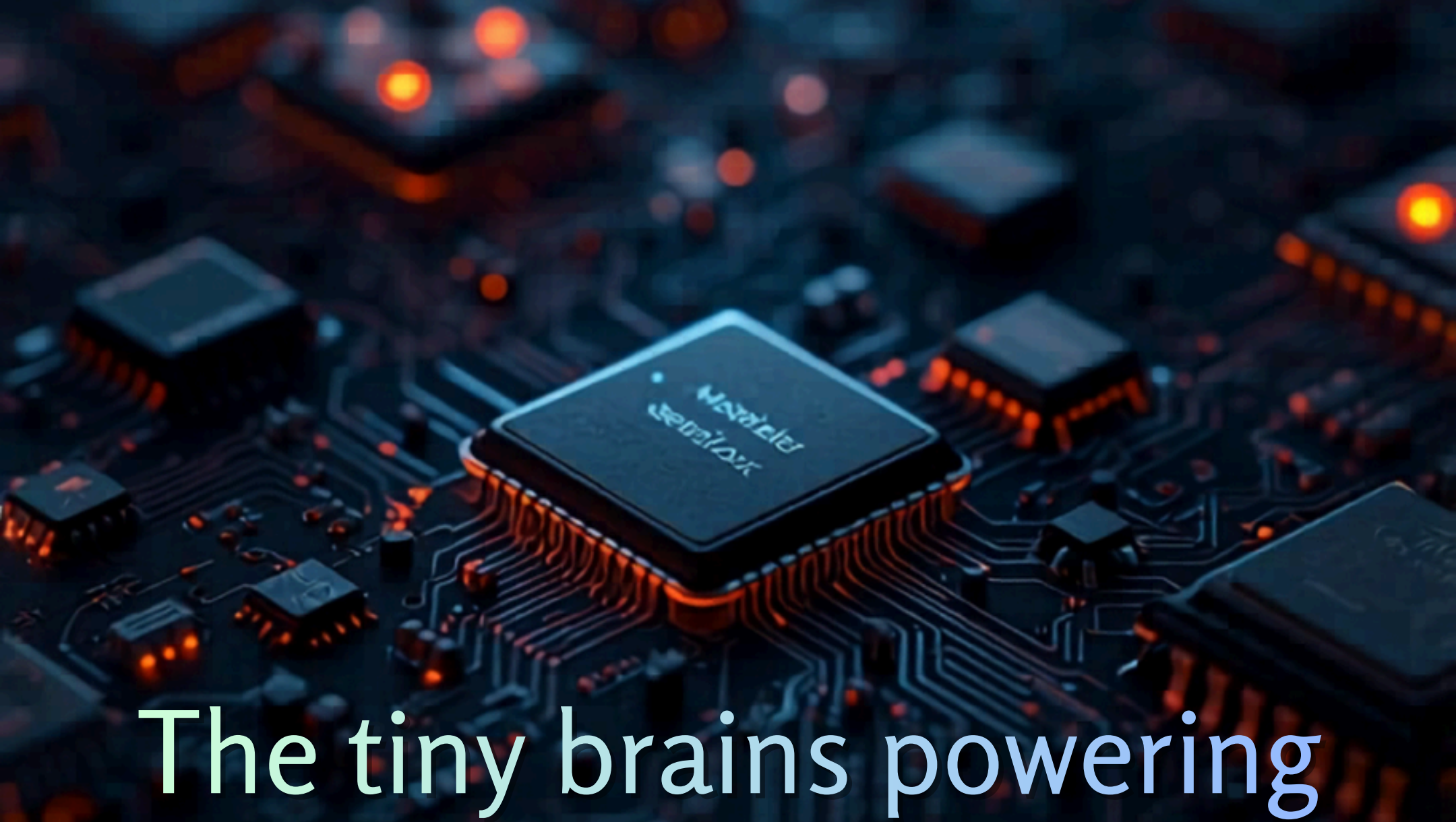
GETTING STARTED WITH ESP32



The tiny brains powering tomorrow's smart world

LinkedIn

Github

ESP32 Lab

From Zero to IoT

A hands-on beginner-to-real-world lab by Aditi Das

What is a Microcontroller?

A microcontroller (MCU) is a tiny self-contained computer on a single chip — it has a processor, memory, and programmable input/output pins all built in. Unlike a regular computer that runs an operating system and many programs at once, a microcontroller runs one program in a loop, responding to the physical world in real time. They power everything around you — the button in your elevator, the thermostat on your wall, your car's dashboard, and every smart home device you own.

What Can Microcontrollers Do?

Read Sensors	Temperature, humidity, motion, light, pressure, GPS — any physical signal.	**Control Actuators**	Drive motors, relays, servos, LEDs, buzzers, and solenoids.
Communicate	Talk over Wi-Fi, Bluetooth, I2C, SPI, UART, MQTT, and more.	**Process & Decide**	Run logic, filter data, trigger actions — all without a cloud.
Log Data	Store readings to SD cards, EEPROM, or send to Google Sheets.	**Go Wireless**	Build mesh networks, remote sensors, or web-controlled devices.

Why ESP32 Specifically?

Out of hundreds of microcontrollers available, the ESP32 stands out because it packs Wi-Fi and Bluetooth directly onto the chip — no external modules needed. It runs at 240MHz with dual cores, has 34 programmable GPIOs, and costs under $5. It is the chip behind countless commercial IoT products and is the go-to starting point for anyone learning embedded systems today.

Real-World Things Built with ESP32

Domain	Example Project
Home Automation	Smart switch, motion-triggered lights, door sensors
Environmental	Temperature & humidity logger pushing to Google Sheets
Security	Door sensor with instant Telegram alert
Display & UI	OLED weather station, web dashboard on local IP
Motor & Robotics	RC car, servo arm controlled via browser
Mesh Networking	Multi-node sensor grid using ESP-NOW, no router needed
Data Logging	SD card + RTC timestamped sensor readings

ESP32 Lab

From Zero to IoT

What is the ESP32?

A low-cost microcontroller with built-in Wi-Fi and Bluetooth — dual-core 240MHz, 520KB RAM, and enough GPIOs to interface with almost any sensor or peripheral.

Board	Notes
ESP32 DevKit	38 pins, more GPIOs, dual-core 240MHz
NodeMCU ESP32-S	Narrower, breadboard-friendly, same chip

GPIO Pin Layout

Click the image to view the full project on GitHub

Pin Types — Quick Reference

Type	Pins	Use
Digital I/O	Most GPIOs	LED, button, relay, sensor ON/OFF
Analog (ADC)	GPIO 32-39	Read voltage — sensors, potentiometers
PWM	Almost any GPIO	Dim LEDs, servo motors
DAC	GPIO 25, 26	True analog voltage output
I2C	GPIO 21 (SDA), 22 (SCL)	OLED, BMP280, MPU6050
SPI	GPIO 18/19/23/5	SD cards, TFT displays
UART	GPIO 1 (TX), 3 (RX)	Serial comms, GPS, Bluetooth serial
Touch	GPIO 4,13,14,15,27,32,33	Capacitive touch buttons
Strapping	GPIO 0, 2, 12, 15	Affect boot — avoid at startup

GPIO 34-39 are input-only (no pull-up). GPIO 6-11 tied to flash — do NOT use.

ESP32 Lab

From Zero to IoT

Wi-Fi Operating Modes

STA Mode (Station)	ESP32 connects to your home router. Gets an IP via DHCP. Use for cloud data, NTP sync, web APIs.
AP Mode (Access Point)	ESP32 becomes the router. Fixed IP: 192.168.4.1. No internet needed — connect phone, open browser.
AP + STA (Dual Mode)	Both simultaneously. Stay online while hosting a local AP — great as a bridge or local dashboard.

Sketch Index

#	Folder	What it does	New concept
01	blink_led	Toggle onboard LED	GPIO, digital write
02	get_mac	Read device MAC address	Device identity, Serial
03	wifi_sta_mode	Connect to router, print IP	STA, RSSI, reconnect
04	wifi_ap_mode	ESP32 hosts its own Wi-Fi	AP mode, 192.168.4.1
05	ap_sta_dual_mode	AP and STA simultaneously	WIFI_AP_STA, bridge
06	web_server_led	Control LED from browser	HTTP server, HTML over Wi-Fi
07	dht_sensor	Live temp and humidity, JSON endpoint	External sensor, JSON
08	esp_now_peer	Two boards talk, no router	ESP-NOW, struct packets
09	oled_i2c	OLED: IP, signal, uptime	I2C, Adafruit GFX
10	mqtt_broker	Publish data, receive commands	MQTT, pub/sub, Home Assistant

Language & Toolchain Options

All sketches use **C++ (Arduino-flavoured)**. Alternatives below:

Language	Tool	Good for	Tradeoff
C++ (Arduino)	Arduino IDE, PlatformIO	Beginners, huge libraries	Abstracted, less control
C/C++ (ESP-IDF)	VS Code + IDF plugin	Full control, production firmware	Steep curve, verbose
MicroPython	Thonny, mpremote	Python familiarity, prototyping	Slower, less RAM
CircuitPython	Mu Editor	Educational, drag-drop FS	Limited ESP32 support
Lua	NodeMCU firmware	Event-driven scripting	Legacy, less maintained
JavaScript	Moddable SDK, Espruino	JS familiarity	Niche, small community
Rust	esp-hal, esp-idf-hal	Memory safety, modern tooling	Experimental setup